



Scope y this

Programación con JavaScript II



Actividad de Bienestar





Temas

- | | |
|---|---|
| 1 | Manejadores de paquetes y empaquetados de módulos |
| 2 | Nuevos tipos y características |
| 3 | Asincronía |
| 4 | API |
| 5 | Ajax |
| 6 | Scope y This |
| 7 | Transpiladores |
| 8 | Pruebas |





Actividad Guiada

Parte 1

Duración: 75 minutos.



Ejercicio 1: Validación de datos de una clase

Objetivo del ejercicio

Implementar una clase Pelicula en JavaScript que realice validaciones robustas de los datos proporcionados para construir una ficha técnica, utilizando métodos específicos para validar cadenas de texto, números, arreglos, años de estreno, géneros permitidos y calificación. Este ejercicio fortalece habilidades de programación orientada a objetos, manejo de errores, y validación de entradas en estructuras complejas.



Desde **ECMAScript 6 (ES6)**, se introdujo la sintaxis **class**, que hace que trabajar con programación orientada a objetos sea más clara y parecida a otros lenguajes como Java o Python.

```
class Persona {  
  constructor(nombre, edad) {  
    this.nombre = nombre;  
    this.edad = edad;  
  }  
  ↓  
  saludar() {  
    console.log(`Hola, soy ${this.nombre} y tengo ${this.edad} años.`);  
  }  
}
```



Crear una instancia (objeto) a partir de la clase

```
const persona1 = new Persona("Ana", 30);  
persona1.saludar(); // Hola, soy Ana y tengo 30 años.
```

¿Para qué se usan las clases?

- Para organizar el código.
- Para reutilizar lógica y estructuras.
- Para crear múltiples objetos similares sin repetir código.



Requerimientos iniciales

Crear una **clase** llamada **Pelicula** que reciba al instanciarse un objeto con los siguientes datos:

- ID de la película en IMDB
- Título
- Director
- Año de estreno
- País o países de origen
- Géneros
- Calificación en IMDB



Validaciones obligatorias

Todos los campos

- Son obligatorios.



Validaciones por campo

ID IMDB:

- Longitud de 9 caracteres.
- Primeros 2 caracteres deben ser letras.
- Los 7 restantes deben ser números.

Título:

- No debe rebasar los 100 caracteres.

Director:

- No debe rebasar los 50 caracteres.

Año de estreno:

- Número entero de 4 dígitos.



Validaciones por tipo de dato

País o países:

- Deben pasarse como arreglo.

Géneros:

- Deben pasarse como arreglo.
- Cada género debe estar dentro de los géneros aceptados.

Calificación:

- Debe ser un número entre 0 y 10.
- Puede tener solo un decimal.



Géneros aceptados

Acción, Adultos, Animación, Autobiografía, Aventura, Ciencia ficción, Cine negro, Comedia, Concursos de televisión, Crimen, Deportes, Documental, Drama, Familiar, Fantasía, Guerra, Historia, Musicales, Misterio, Noticias, Películas de vaqueros, Programa de entrevistas, Reality shows, Romance, Suspenso, Terror.



Métodos de la clase

1. Método estático `generosAceptados()`

Muestra los géneros válidos.

2. Método `fichaTecnica()`

Muestra toda la información de la película en consola.

3. Métodos de validación internos

(ej. `validarTitulo`, `validarNumero`, etc.)



Preparación del entorno

1. Crea un **archivo** `index.html` en tu directorio de trabajo.
2. Dentro de ese mismo directorio, crea una **carpeta** llamada `js`.
3. Dentro de `js`, crea el **archivo** `Pelicula.js`.
4. Enlaza `Pelicula.js` al HTML usando la etiqueta `<script src="js/Pelicula.js"></script>` al final del body.



Crear la clase Pelicula

- En Pelicula.js, crea la clase con el nombre Pelicula.
- Dentro de la clase, define el constructor que reciba un objeto con estas propiedades:

```
{id, titulo, director, estreno, pais, generos, calificacion}  
class Pelicula{  
  constructor({id, titulo, director, estreno, pais, generos, calificacion}){  
  
  }  
  
  validarCadena(propiedad, valor){  
  
  }  
}
```



Asignar propiedad dentro del constructor

Usa `this.propiedad = valor` para asignar cada propiedad del objeto recibido

```
class Pelicula{  
    constructor({ id, titulo, director, estreno, pais, generos, calificacion }) {  
        this.id = id;  
        this.titulo = titulo;  
        this.director = director;  
        this.estreno = estreno;  
        this.pais = pais;  
        this.generos = generos;  
        this.calificacion = calificacion;  
    }  
}
```

`this` hace referencia al **objeto que ejecuta la función**. Su valor depende de **cómo se llama** a la función, no de dónde está definida.



Validación de cadena de texto

Crea una función **validarCadena(propiedad, valor)** que haga lo siguiente:

1. Si el valor está vacío: `if (!valor) {`
`console.warn(`${propiedad} "${valor}" está vacío`);`
`return false;`
2. Si el valor no es una cadena: `if (typeof valor !== "string") {`
`console.error(`${propiedad} "${valor}" no es una cadena de texto`);`
`return false;`
3. Si pasa ambas validaciones, retorna true.
`return true;`



Validar ID de IMDB

```
const regex = /^[a-z]{2}([0-9]{7})$/;
```

Crea el método **validarIMDB(id)** dentro de la clase:

1. Llama a `validarCadena('IMDB id', id)`

2. Si retorna true, valida con expresión regular:

```
if (!(/^[a-z]{2}([0-9]{7})$/).test(id)) {  
  console.error(`IMBD id ${id} no es válido. Debe tener nueve  
caracteres: los dos primeros letras minúsculas y los siete restantes  
números`);  
}
```

3. Llama a este método dentro del constructor.



Validar ID de IMDB

```
class Pelicula{
  constructor({ id, titulo, director, estreno, pais, generos, calificacion }) {
    ...

    this.validarIMDB(id); ←

  }

  // Validación de cadenas con longitud máxima
  ...

  validarIMDB(id) {
    if (this.validarCadena("IMDB ID", id)) {
      const regex = /^( [a-z]{2}) ([0-9]{7})$/;

      if (!regex.test(id)) {
        console.error(`IMDB ID "${id}" no es válido, debe tener nueve
caracteres: los dos primeros letras minúsculas y los siete restantes números.`);
      }
    }

    return true;
  }
}
```



Instanciación para probar la clase

Crea una constante para probar la clase:

```
> class Pelicula{ ...  
}  
  
const peli = new Pelicula({  
  id: "tt1234567",  
  titulo: "Nombre de la película",  
  director: "Nombre del director",  
  estreno: 2020,  
  pais: ["México"],  
  generos: ["Drama"],  
  calificacion: 8.5  
});
```



Validar longitud de cadena

Crea el método `validarLongitudCadena (propiedad, valor, longitud)` que:

- Compruebe si **`valor.length > longitud`**



Validar longitud de cadena

```
class Pelicula{
  constructor({ id, titulo, director, estreno, pais, generos, calificacion }) {...}

  // Validación de cadenas con longitud máxima
  validarCadena(propiedad, valor) {...}

  // Validación IMDB ID
  validarIMDB(id) {...}

  validarLongitudCadena(propiedad, valor, longitud) {
    if (valor.length > longitud) {
      console.error(`${propiedad} "${valor}" excede el número de caracteres permitidos (${longitud})`);
      return false;
    }
    return true;
  }
}
```



Validar Título

- Valida que el título:
 - Sea una cadena de texto válida.
 - No exceda los **100 caracteres** de longitud.
- Si ambas validaciones se cumplen, retorna true.
- Si no, muestra un error en consola.

```
validarTitulo(titulo) {  
    if (this.validarCadena("Título", titulo)) {  
        if (this.validarLongitudCadena("Título", titulo, 100)) {  
            return true;  
        }  
    }  
    console.error(`Título "${titulo}" no es válido`);  
    return false;  
}
```



Validar Director

- Valida que el director:
 - Sea una cadena de texto válida.
 - No exceda los **50 caracteres** de longitud.
- Si ambas validaciones se cumplen, retorna true.
- Si no, muestra un error en consola.

```
validarDirector(director) {  
    if (this.validarCadena("Director", director)) {  
        if (this.validarLongitudCadena("Director", director, 50)) {  
            return true;  
        }  
    }  
    console.error(`Director "${director}" no es válido`);  
    return false;  
}
```



Constructor

```
constructor({ id, titulo, director, estreno, pais, generos, calificacion }) {  
  this.id = id;  
  this.titulo = titulo;  
  this.director = director;  
  this.estreno = estreno;  
  this.pais = pais;  
  this.generos = generos;  
  this.calificacion = calificacion;  
  
  this.validarIMDB(id);  
  this.validarTitulo(titulo);  
  this.validarDirector(director);  
}
```



Método validarNumero(propiedad, valor)

1. Valida que un **valor numérico** (como el año de publicación o calificación):
 - No esté vacío.
 - Sea de tipo numérico
2. Si cumple con ambas condiciones, retorna true.



```
validarNumero(propiedad, valor) {  
  if (!valor) {  
    console.warn(`${propiedad} "${valor}" está vacío`);  
    return false;  
  }  
  
  if (typeof valor !== "number") {  
    console.error(`${propiedad} "${valor}" ingresado no es un número`);  
    return false;  
  }  
  
  return true;  
}
```



Validar año de Estreno

Validar que el **año de estreno**:

1. Sea un **número válido**.
2. Tenga exactamente **4 dígitos numéricos**.

```
validarEstreno(estreno) {  
  if (this.validarNumero("Año de estreno", estreno)) {  
    if (!/^\d{4}$/.test(estreno)) {  
      console.error(`Año de estreno "${estreno}" no es válido,  
                    debe ser un número de 4 dígitos`);  
      return false;  
    }  
    return true;  
  }  
  return false;  
}
```



Receso

Duración: 10 minutos.



Actividad Guiada

Parte 2

Duración: 75 minutos.



Método validarArreglo(propiedad, valor)

Validar que el valor de una propiedad como **países** o **géneros**:

1. Sea un **arreglo no vacío**
2. Todos sus elementos sean **cadenas de texto**

Retorna `true` si todas las validaciones se cumplen.



Método validarArreglo(propiedad, valor)

```
validarArreglo(propiedad, valor) {  
  if (!valor) {  
    console.warn(`${propiedad} "${valor}" está vacío`);  
    return false;  
  }  
  
  if (!Array.isArray(valor)) {  
    console.error(`${propiedad} "${valor}" no es un arreglo`);  
    return false;  
  }  
  
  if (valor.length === 0) {  
    console.error(`${propiedad} "${valor}" no tiene elementos`);  
    return false;  
  }  
  
  for (let elemento of valor) {  
    if (typeof elemento !== "string") {  
      console.error(`${propiedad} contiene un elemento no válido: "${elemento}"  
        |         |         |         |  
        no es una cadena de texto`);  
      return false;  
    }  
  }  
  
  return true;  
}
```



Validar País

Validar que la propiedad **pais** sea un **arreglo válido** utilizando el método `validarArreglo`.

```
validarPais(pais) {  
    return this.validarArreglo("Pais", pais);  
}
```

Este método debe ser llamado **desde el constructor** para validar automáticamente al crear la instancia.

```
class Pelicula{  
    constructor({ id, titulo, director, estreno, pais, generos, calificacion }) {  
        ...  
        this.validarPais(pais);  
    }  
}
```



Métodos Estáticos

¿Qué es un método estático?

- Es un **método que pertenece a la clase, no a una instancia.**
- Se invoca directamente desde la clase, **sin crear un objeto.**

¿Para qué sirve?

- Acceder a **funciones o datos comunes a todas las instancias**, como:
 - Constantes o listas compartidas (listaGeneros)
 - Métodos de utilidad o validaciones generales



Métodos Estáticos vs Métodos Instanciados

Método Estático	Método Instanciado
Se usa sin crear una instancia	Requiere crear un objeto
Menor consumo de memoria (no instancias)	Cada objeto consume memoria
Ideal para funciones auxiliares	Ideal para manejar datos específicos del objeto
Más rápido de ejecutar	Puede ser más flexible



Método estático para géneros aceptados

1. Declaración de atributos estático `listaGeneros`.

```
static listaGeneros = [  
  "Acción", "Adultos", "Animación", "Autobiografía", "Aventura",  
  "Ciencia ficción", "Cine negro", "Comedia", "Concursos de televisión", "Crimen",  
  "Deportes", "Documental", "Drama", "Familiar", "Fantasía",  
  "Guerra", "Historia", "Musicales", "Misterio", "Noticias",  
  "Películas de vaqueros", "Programa de entrevistas", "Reality shows",  
  "Romance", "Suspenso", "Terror"  
];
```

2. Crear Método estático `generosAceptados()`

```
static generosAceptados() {  
  console.info(`Los géneros aceptados son: ${Pelicula.listaGeneros.join(", ")}`);  
}
```

3. Invocar el método sin crear instancia



Validar Genero

1.  **Validar** que sea un arreglo

Se usa la función `validarArreglo("Géneros", generos)`

Si **no es válido**, se detiene el flujo.

2.  **Recorrer** el arreglo con *for...of* para cada `genero`:

- Verifica si está en la lista

estática `Pelicula.listaGeneros` con `.includes()`

3.  Si no se encuentra:

- Muestra error en consola:

`Error: Género "X" no es aceptado. Géneros recibidos: A, B, C`

- Luego imprime:

`Pelicula.generosAceptados(); // Muestra géneros válidos`



Validar Genero

```
validarGeneros(generos) {  
  if (this.validarArreglo("Géneros", generos)) {  
    for (let genero of generos) {  
      if (!Pelicula.listaGeneros.includes(genero)) {  
        console.error(  
          `Género "${genero}" no es aceptado.  
          Géneros recibidos: ${generos.join(", ")}`  
        );  
        Pelicula.generosAceptados();  
      }  
    }  
  }  
}
```



Validar Calificación con rango y decimales

1. Validar que sea número

2. Validar rango 0 a 10

Si calificacion < 0 || calificacion > 10, imprimir error:

```
console.error(`La calificación "${calificacion}" debe estar en un  
rango entre 0 y 10.`);
```

3. Limitar a 1 decimal

Si la validación pasa, redondear:

```
this.calificacion = parseFloat(calificacion.toFixed(1));
```



Validar Calificación con rango y decimales

```
validarCalificacion(calificacion) {  
  if (this.validarNumero("Calificación", calificacion)) {  
    if (calificacion < 0 || calificacion > 10) {  
      console.error(`La calificación "${calificacion}"  
                    debe estar en un rango entre 0 y 10.`);  
    } else {  
      this.calificacion = parseFloat(calificacion.toFixed(1));  
    }  
  }  
}
```



Método fichaTecnica()

```
fichaTecnica() {  
  console.info(  
    `📄 Ficha Técnica:  
    ID: ${this.id}  
    Título: ${this.titulo}  
    Director: ${this.director}  
    Año de estreno: ${this.estreno}  
    País(es): ${this.pais.join(", ")}  
    Género(s): ${this.generos.join(", ")}  
    Calificación: ${this.calificacion}`  
  );  
}
```



Invocación método fichaTecnica()

```
const pelicula = new Pelicula({  
  id: "tt1234568",  
  titulo: "Inception",  
  director: "Christopher Nolan",  
  estreno: 2010,  
  pais: ["Estados Unidos", "Reino Unido"],  
  generos: ["Acción", "Ciencia ficción", "Suspenso"],  
  calificacion: 8.8  
});  
  
pelicula.fichaTecnica();
```



Generar tres instancias de manera automatizada

1. Se define un arreglo `peliculasData` con 3 objetos de películas.

```
const peliculasData = [  
  {  
    id: "tt1234567",  
    titulo: "Inception 2",  
    director: "Christopher Nolan",  
    estreno: 2011,  
    pais: ["Estados Unidos", "Reino Unido"],  
    generos: ["Acción", "Ciencia ficción", "Suspenso"],  
    calificacion: 8.8  
  },  
  {  
    id: "tt2345678",  
    titulo: "The Matrix",  
    director: "Lana Wachowski",  
    estreno: 1999,  
    pais: ["Estados Unidos"],  
    generos: ["Acción", "Ciencia ficción"],  
    calificacion: 8.7  
  },  
  {  
    id: "tt3456789",  
    titulo: "Interstellar",  
    director: "Christopher Nolan",  
    estreno: 2014,  
    pais: ["Estados Unidos", "Reino Unido"],  
    generos: ["Aventura", "Drama", "Ciencia ficción"],  
    calificacion: 8.6  
  }  
];
```



Generar tres instancias de manera automatizada

2. Se recorre el arreglo con `forEach`, creando una nueva instancia `Pelicula` con cada objeto.
3. Se llama al método `fichaTecnica()` para mostrar los datos de cada película.

```
peliculasData.forEach(peliculaData => {  
  const pelicula = new Pelicula(peliculaData);  
  pelicula.fichaTecnica();  
});
```



Cierre

Duración: 10 minutos.



¿Qué pusimos en práctica?

- Creación y uso de clases en JavaScript.
- Declaración de métodos estáticos y de instancia.
- Validaciones de:
 - Cadenas
 - Números (enteros y decimales)
 - Arreglos
 - Longitudes
- Control de errores mediante `console.warn()` y `console.error()`



Repasemos

Scope (alcance) en clases:

- Cada método actúa **dentro del contexto del objeto**, accediendo a propiedades y a otros métodos de forma controlada.

Uso de `this`:

- Para acceder a propiedades:

`this.titulo, this.director, this.generos, etc.`

- Para invocar métodos internos desde el constructor o desde otros métodos:

`this.validarTitulo(titulo), this.validarGeneros(generos)`